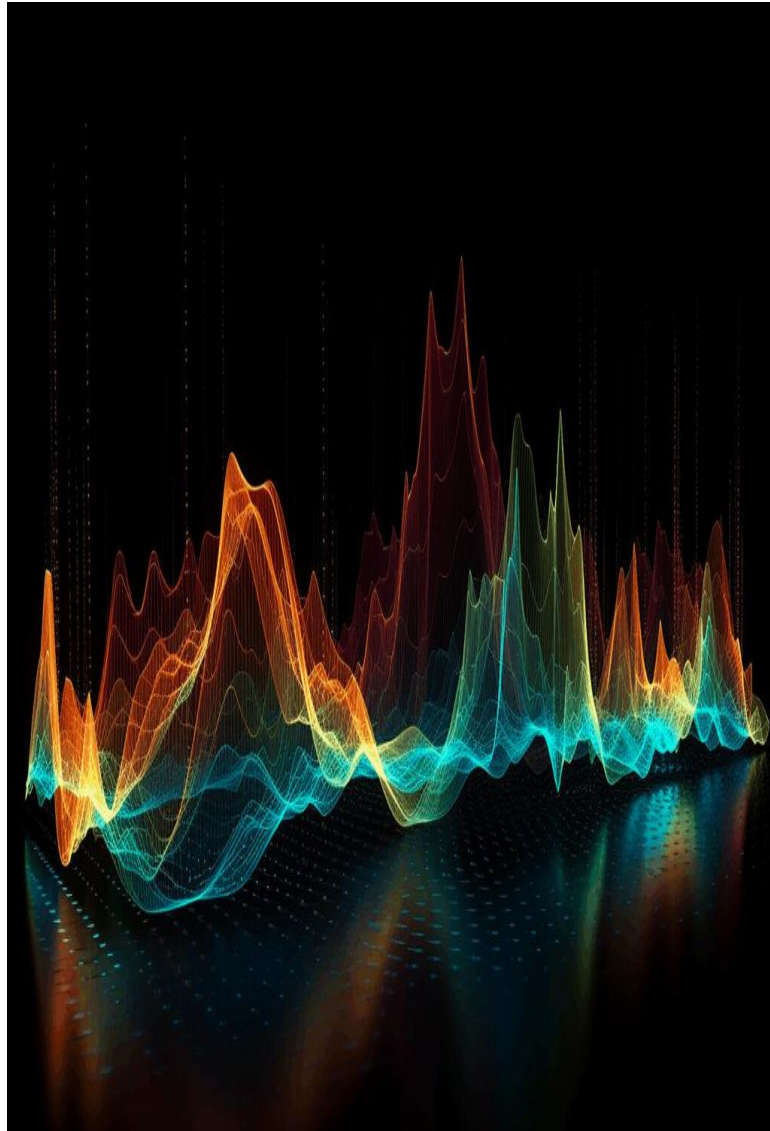




Applied Numerical Methods

WITH OCTAVE

LAB1 SUPPORT

ENSIA | 3rd Year | Spring -- 2024

PART I : Introduction

Why dedicated languages for mathematical computation ?

The answer to this question is simple : it is to spare programmers from exerting too much effort on the technical aspects of programming, allowing them to focus on mathematical problems. The first programming language that could be considered advanced was created to transcribe mathematical formulas. This language is FORTRAN (FORMula TRANslator). However, this language, like other general-purpose languages such as Algol, C, or Pascal, is not close to the mathematical language. This is why even more advanced languages dedicated to mathematics have been created.

Choosing a Language

The list of all programming languages is extensive (currently over 1000). Making a choice is often not straightforward. The fundamental question you need to ask yourself is: 'What type of problem do I want to solve?' This will guide you towards one group of languages or another. Indeed, there are general-purpose languages like Python, C, Pascal, C++, Java, and there are more specialized languages such as Matlab, R, Octave, Scilab, JavaScript, HTML, SQL, PHP, etc.

In the field of mathematical computation, we can find several languages : TkSolver, Matlab, Scilab, SageMath, Analytica, FreeMat, and many others!

Why Octave?

Matlab is considered by many programmers as the benchmark in the field of scientific programming. Indeed, the Institute of Electrical and Electronics Engineers (IEEE), in its annual report on the best programming languages for the year 2023, ranks it in the Top 20 for the development of enterprise, desktop, and scientific applications. It is surpassed only by general-purpose languages like Java and Python, and a single language specialized in the field of statistics, which is R.

In order to benefit from Matlab for free, we will introduce, in this course, the Octave language, which is the closest clone to Matlab software.

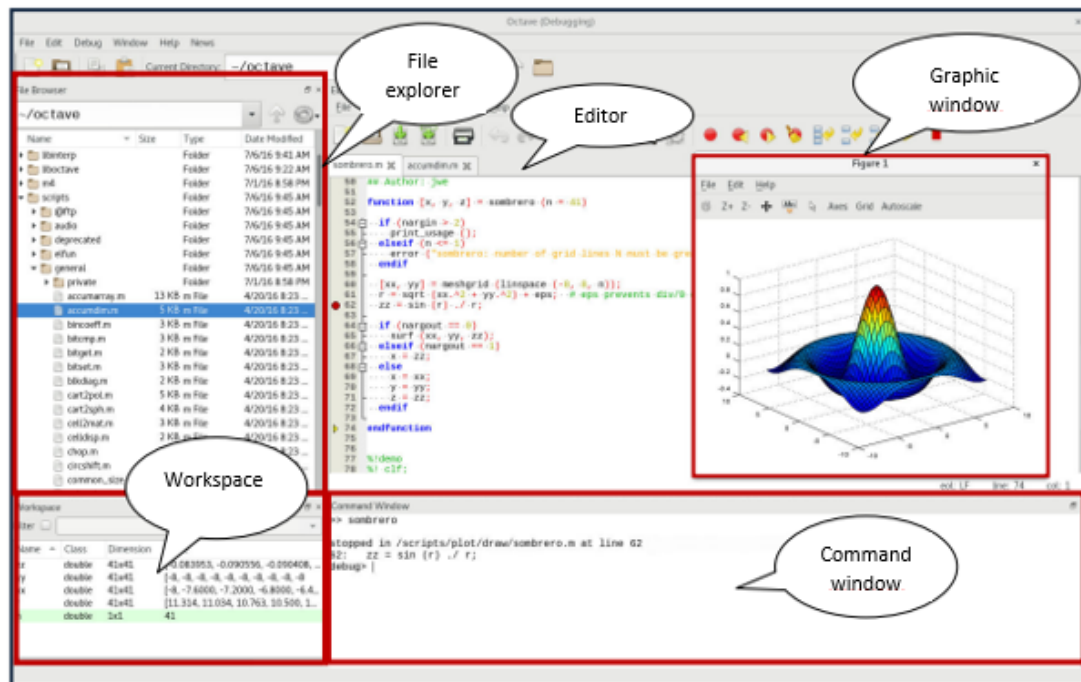
Installation

Octave is free, open-source, and cross-platform. You can obtain it by visiting its official website: <https://octave.org/download.html> . Its installation is very simple. Under Linux, you follow the usual software installation procedure using a package manager for your distribution (apt for Debian and yum for Redhat and Fedora, for example, under Ubuntu (Debian), simply type the following command in the terminal : '**sudo apt-get install octave**'. You can, of course, use a graphical package manager like Synaptic or the Ubuntu Software Center to achieve the same.

On Windows, the process is no more complicated. You just need to download the executable code of the latest version of Octave from its official website, then run this file and follow the installation procedure, making a few minor configurations such as specifying the location (disk folder) where you want to install Octave.

Graphical Interface

When you launch octave, a graphical interface appears. It consists of several windows: Commands, Command History, File Explorer, Editor, Workspace, and Documentation. It is also possible to open graphic windows.



PART II : Getting started

1. TYPES OF NUMBERS

There are three types of numbers: **real** numbers, **integers**, and **complex** numbers.

- **Real Numbers:** There are two representations: 'double' and 'single.' Internally, Octave defaults to storing all numbers in double-precision floating-point format (in IEEE format, occupying 8 bytes per number, hence 64 bits). The numbers have a finite precision of 16 significant decimal digits and a range from 10^{-308} to 10^{308} .

Here are examples of valid real numbers:

5, -39, 0.000365, -1.6341e11, 4.531e-15

It is possible to explicitly reduce the precision of numbers using the 'single()' function. It is also possible to convert a number represented in single precision to double precision using the 'double()' function.

- **Integer numbers :** We have just seen that Octave handles numbers by default in floating-point real form (double precision or, upon request, single precision). Thus,

the expression 'n = 432' internally stores the specified number 'n' as a double-precision real variable, even though an integer was entered.

- **Complex Numbers:** Octave handles complex numbers. These numbers are internally stored in the form of real 'double precision' numbers on 2x 8 bytes. The first 8 bytes represent the real part, and the second 8 bytes represent the imaginary part. Here are some examples of valid representations of complex numbers:

$4e-13 - 5.6i$	$-45 + 5*j$	$3 + i$
$3 - j$	$(14.5+5) + 13j$	$13+14 + 3i$

2. VARIABLES AND EXPRESSIONS

Variables: The variables created during an Octave working session (interactively or from a script) reside in memory in what is called the 'workspace.' There is no need to declare a variable ; Octave creates it as soon as it encounters its name on the left side of an assignment. It deduces its type and the space to allocate in the workspace during the evaluation of the expression found on the right side of an assignment.

Expressions: An Octave 'expression' is a valid construction that uses numbers, variables, operators, and functions. Examples: pi^2 , $\text{sqrt}((b^2)-(4a*c))$

- If an expression is written without specifying a receiving variable name, Octave creates a default variable that it names 'ans' (ans for answer).
- 'who' and 'whos' allow listing variables
- 'clear' is used to delete variables"

Command « format » :

- Format **long** : displays numbers with 14 digits after the decimal point
- Format **short** : displays numbers with 4 digits after the decimal point
- Format **bank** : displays numbers with 2 digits after the decimal point
- Format **rat** : displays numbers in the form of a fraction a/b

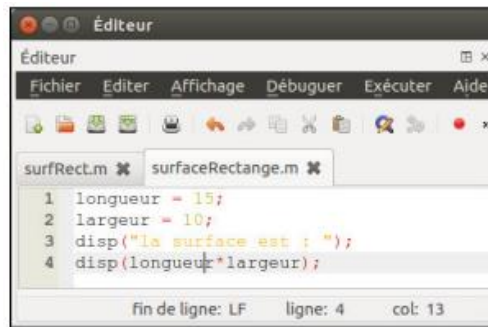
3. SCRIPTS

These are sequences of commands saved on disk as files with the extension '.m'. These files are called 'm-files'. To reuse them, simply specify the name of the 'm-file' from a command window or from a function or another script.

The script should be accessible, meaning it must be in the current directory, or you need to provide the full path to the file of this script.

Example : With the Octave editor, you can enter a script to calculate the area of a rectangle:

To execute this script, simply type its name in the command window.



4. FUNCTIONS

Functions are what make Octave rich and powerful. As it is a language created for scientific computation, it provides a large number of predefined mathematical functions, but users can also create their own. Functions follow the same naming rules as variables.

Octave provides a wide list of predefined functions (in appendix)

5. RANGES :

A range is a convenient way to describe a row vector composed of uniformly spaced values. The expression of a range is defined by an initial value, an optional value defining the step between two successive values, and a maximum value. These three values are separated by the colon operator ':'. If the step is not given, it is assumed to be equal to 1.

Examples:

- '1:5' defines a row vector composed of values starting from 1 and ending at 5 with a step of 1, resulting in: **[1, 2, 3, 4, 5]**.
- The expression '1:2:5' defines the vector composed of the following values: **[1, 3, 5]**.

6. VECTORS

Octave does not distinguish between a scalar, a vector, or a matrix. Thus, a vector is nothing more than a matrix with a single row and multiple columns ($1 \times M$) or a single column and multiple rows ($N \times 1$). In the first case ($1 \times M$), it is referred to as a 'row vector.' In the second case ($N \times 1$), it is referred to as a 'column vector.'

The elements of vectors are numbered with integers starting from the value 1 (not 0, as in most other programming languages). Creating a 'row vector': Simply specify values within square brackets, separated by spaces or commas. For example, if we want to create the vector $v1=(1,5,7,8)$, we write the following command:

$v1 = [1 \ 5 \ 7 \ 8]$ Or $v1 = [1, 5, 7, 8]$

Creating a 'column vector': Simply specify values within square brackets, separated by semicolons or by pressing the 'ENTER' key. For example, if we want to create the vector $v2=(1,15,10,8)$, we write the following command:

$v1 = [1 ; 15 ; 10 ; 8]$

How to access the elements of a vector?

This is done by specifying two pieces of information : the name of the vector followed, in parentheses, by indices : **vect(indices)**

To determine the size of a vector: Use the 'length' function. For example, 'length(v)' returns the number of elements in v.

7. MATRICES

A matrix is a rectangular array with 2 dimensions of $N \times M$ elements (N rows and M columns) of integer, real, complex numbers, or characters. Similar to vectors, creating a matrix is done by assigning elements enclosed in square brackets. The elements of rows are separated by semicolons, and those of columns are separated by spaces or commas.

Examples: A matrix with 2 rows and 3 columns having values: [-2 -1 0 ; 4 3 3]

How to access the elements of a matrix?

It is similar to mathematics: You specify the name of the matrix followed by parentheses containing two indices, where the first indicates the row number and the second indicates the column number.

There are several Octave functions for creating specific matrices: transpose matrix, diagonal matrix, triangular matrix, unitary matrix, zero matrix, etc.

Some examples:

- **ones(n)**: creates a square matrix of size nxn with elements initialized to '1'.
- **ones(n,p)**: creates a matrix with n rows and p columns with elements initialized to '1'.
- **zeros(n)**: creates a square matrix of size nxn with elements initialized to '0'.
- **zeros(n,p)**: creates a matrix with n rows and p columns with elements initialized to '0'.
- **eye(n)**: creates a unit square matrix of size nxn.
- **eye(n,p)**: creates a unit matrix with n rows and p columns.
- **diag(vec)**: Applied to a row or column vector 'vec', this function returns a square matrix with the elements of 'vec' on its main diagonal and other elements equal to '0'.
- **diag(mat)**: Applied to a matrix, it returns, in a column vector, the elements of the main diagonal of that matrix.

PART III : Programming with Octave

Octave allows the users to define their own functions and new scripts, thus extending the offered functionalities. It is indeed a full-fledged programming language!

1. **DISPLAYING TEXT AND VARIABLES :** The 'disp' and 'printf' commands allow displaying variables or text : **disp(variable)**
2. **INPUT OF INFORMATION FROM THE KEYBOARD :** The 'input' command allows the user to input data from the keyboard. This command can be used in two ways:

- **input("message", "s")**
- **input("message")** The first form displays a message on the screen and retrieves the user's input as a string. The second form allows the user to enter data of various types, such as scalars, vectors, or matrices."

3. CONTROL STRUCTURES

Like all other programming languages, Octave provides commands for conditional or repetitive processing.

A. Conditions

The basis for controlling the flow of commands in a program is the evaluation of conditions. A condition is an expression that can be true or false. It is composed using relational operators and logical operators.

- B. **Relational Operators** These operators allow for numerical tests by constructing 'logical expressions,' i.e., expressions that return true or false values.

Opérateur	Description
==	Test d'égalité
~=	Test d'inégalité
!=	
<	Tes d'infériorité
>	Test de supériorité
>=	Test de supériorité ou égalité
<=	Tes d'infériorité ou égalité

C. Logical operators

Logical operators take logical expressions as arguments and return logical values true (1) or false (0).

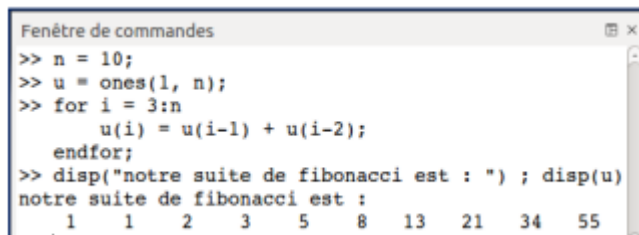
Opérateur	Description
~ expression	Négation logique
Exp1 & Exp2	Et logique entre Exp1 et Exp2
Exp1 && Exp2	Et logique entre Exp1 et Exp2, mais plus efficace que le simple &
Exp1 Exp2	Ou logique entre Exp1 et Exp2
Exp1 Exp2	Ou logique entre Exp1 et Exp2, mais plus efficace que le simple

D. The command 'for'

Syntax:

```
for var = expression  
  
    Instructions  
  
endfor
```

Example :

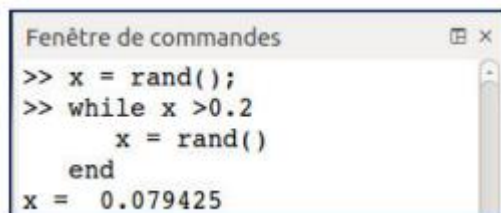


```
Fenêtre de commandes  
>> n = 10;  
>> u = ones(1, n);  
>> for i = 3:n  
    u(i) = u(i-1) + u(i-2);  
endfor;  
>> disp("notre suite de fibonacci est : ") ; disp(u)  
notre suite de fibonacci est :  
    1    1    2    3    5    8   13   21   34   55
```

E. The 'while' command

It allows repeating a process as long as a condition is satisfied.

Example :



```
Fenêtre de commandes  
>> x = rand();  
>> while x > 0.2  
    x = rand();  
end  
x = 0.079425
```

F. The command if

Syntax :

```
if condition 1  
  
    Instructions if condition 1 True  
  
elseif condition 2 Instructions if condition 2 True  
  
else Instructions if all conditions false  
  
end
```

Example :


```
Fenêtre de commandes
>> age = input("donnez votre âge : ")
donnez votre âge : 12
age = 12
>> if age < 12
    disp("vous êtes un enfant")
elseif age < 18
    disp("vous êtes un adolescent")
elseif age < 65
    disp("vous êtes un adulte")
else
    disp("vous êtes un vieux!")
end
vous êtes un adolescent
```

G. The 'switch' command

Allows executing instructions based on the values of a certain variable.

Syntax:

```
switch expression

    case value1

        instructions if expression is value1

    case value2

        instructions if expression is value2

    .
    .
    .

    otherwise instructions

end
```

Example :

```
Fenêtre de commandes
>> couleurs = ["Vert"; "Rouge"; "Bleu"; "Jaune"];
>> indice = floor(rand()*4)+1;
>> switch couleurs(indice)
    case "Vert"
        disp("j'ai tiré au hazard le Vert")
    case "Rouge"
        disp("j'ai tiré au hazard le Rouge")
    case "Bleu"
        disp("j'ai tiré au hazard le Bleu")
    otherwise
        disp("j'ai tiré au hazard le Jaune")
    end
j'ai tiré au hazard le Jaune
```

4. FUNCTIONS

Functions allow code reuse and help avoid redundancies. In Octave, the 'function' command is used to declare a function. Functions, in general, are identified by a name, require zero or more parameters, and can return values (scalars, vectors, or matrices, for example).

Syntax:

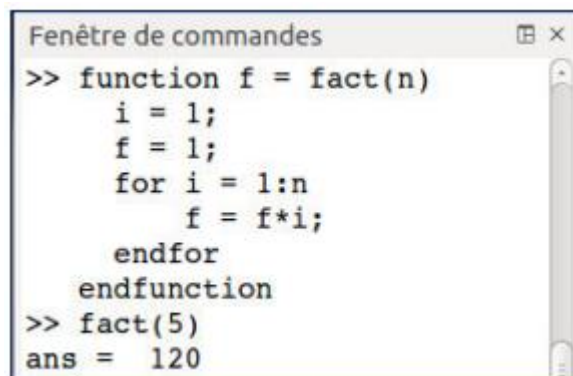
```
function OUT = FonctionName(IN)

    Function body
```

With:

- **OUT**: the output parameters, which can be empty, a scalar, a vector, a matrix, etc.
- **IN**: the input parameters, which can be empty, a scalar, a vector, a matrix, etc.
- The body of the function consists of Octave instructions.

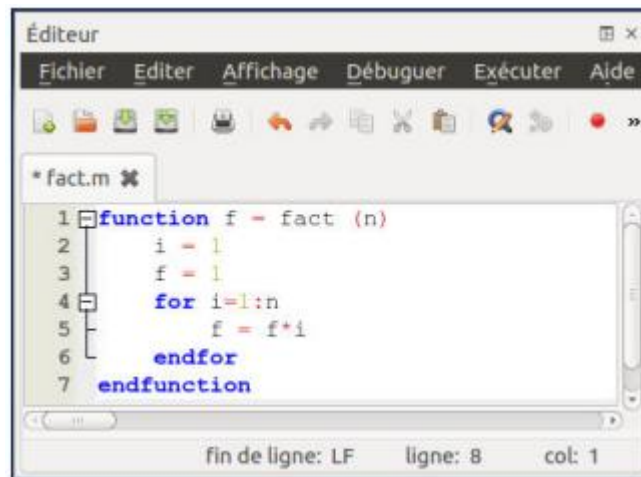
Example : Function to compute the factorial of a number.



```
Fenêtre de commandes
>> function f = fact(n)
        i = 1;
        f = 1;
        for i = 1:n
            f = f*i;
        endfor
    endfunction
>> fact(5)
ans = 120
```

5. SCRIPTS

A script file contains instructions that are read and executed sequentially by the Octave interpreter. These files must be in text format and are recognizable by their '.m' extension. Octave's graphical environment provides an editor for editing scripts, but you can edit these scripts with any text editor. Note that a function can also be saved in a script file. In the example below, I created a function in a file named 'fact.m'.



```
1 function f = fact (n)
2     i = 1
3     f = 1
4     for i=1:n
5         f = f*i
6     endfor
7 endfunction
```

fin de ligne: LF ligne: 8 col: 1

Note : In the above case, the name of the function and the name of the script file are the same.

APPENDIX : PREDEFINED FUNCTIONS IN OCTAVE

Fonction	Description
<code>sqrt(var)</code>	Racine carrée de <i>var</i> . Remarque : pour la racine <i>n</i> -ème de <i>var</i> , faire <i>var</i> ^(1/ <i>n</i>)
<code>exp(var)</code>	Exponentielle de <i>var</i>
<code>log(var)</code> <code>log10(var)</code> <code>log2(var)</code>	Logarithme naturel de <i>var</i> (de base e), respectivement de base 10 , et de base 2 Ex: <code>log(exp(1)) => 1</code> , <code>log10(1000) => 3</code> , <code>log2(8) => 3</code>
<code>cos(var)</code> <code>acos(var)</code>	Cosinus, resp. arc cosinus, de <i>var</i> . Angle exprimé en radian
<code>sin(var)</code> <code>asin(var)</code>	Sinus, resp. arc sinus, de <i>var</i> . Angle exprimé en radian

Fonction	Description
<code>sec(var)</code> et <code>csc(var)</code>	Sécante, resp. cosécante, de <i>var</i> . Angle exprimé en radian
<code>tan(var)</code> et <code>atan(var)</code>	Tangente, resp. arc tangente, de <i>var</i> . Angle exprimé en radian
<code>cot(var)</code> et <code>acot(var)</code>	Cotangente, resp. arc cotangente, de <i>var</i> . Angle exprimé en radian
<code>atan2(dy,dx)</code>	Angle entre -pi et +pi correspondant à <i>dx</i> et <i>dy</i>
<code>cart2pol(x,y {,z})</code> et <code>pol2cart(th,r {,z})</code>	Passage de coordonnées cartésiennes en coordonnées polaires, et vice-versa
<code>cosh</code> , <code>acosh</code> , <code>sinh</code> , <code>asinh</code> , <code>sech</code> , <code>asch</code> , <code>tanh</code> , <code>atanh</code> , <code>coth</code> , <code>acoth</code>	Fonctions hyperboliques...
<code>factorial(n)</code>	Factorielle de <i>n</i>

rand rand(<i>n</i>) rand(<i>n,m</i>)	Génération de nombres aléatoires réels compris entre 0.0 et 1.0 selon une distribution uniforme standard : - génère un nombre aléatoire - génère une matrice carrée <i>n</i> x <i>n</i> de nombres aléatoires - génère une matrice <i>n</i> x <i>m</i> de nombres aléatoires
fix(<i>var</i>) round(<i>var</i>) floor(<i>var</i>) ceil(<i>var</i>)	Troncature à l'entier, dans la direction de zéro (donc 4 pour 4.7, et -4 pour -4.7) Arrondi à l'entier le plus proche de <i>var</i> Le plus grand entier qui est inférieur ou égal à <i>var</i> Le plus petit entier plus grand ou égal à <i>var</i> Ex: fix(3.7) et fix(3.3) => 3, fix(-3.7) et fix(-3.3) => -3 round(3.7) => 4, round(3.3) => 3, round(-3.7) => -4, round(-3.3) => -3 floor(3.7) et floor(3.3) => 3, floor(-3.7) et floor(-3.3) => -4 ceil(3.7) et ceil(3.3) => 4, ceil(-3.7) et ceil(-3.3) => -3
mod(<i>var1</i>,<i>var2</i>) rem(<i>var1</i>,<i>var2</i>)	Fonction <i>var1</i> "modulo" <i>var2</i> Reste ("remainder") de la division de <i>var1</i> par <i>var2</i> Remarques: - <i>var1</i> et <i>var2</i> doivent être des scalaires réels ou des tableaux réels de même dimension - rem a le même signe que <i>var1</i>, alors que mod a le même signe que <i>var2</i> - les 2 fonctions retournent le même résultat si <i>var1</i> et <i>var2</i> ont le même signe Ex: mod(3.7, 1) et rem(3.7, 1) retournent 0.7, mais mod(-3.7, 1) retourne 0.3, et rem(-3.7, 1) retourne -0.7
idivide(<i>var1</i>, <i>var2</i>, 'regle')	Division entière. Fonction permettant de définir soi-même la règle d'arrondi.
abs(<i>var</i>)	Valeur absolue (positive) de <i>var</i> Ex: abs([3.1 -2.4]) retourne [3.1 2.4]
sign(<i>var</i>)	(signe) Retourne "1" si <i>var</i>>0, "0" si <i>var</i>=0 et "-1" si <i>var</i><0 Ex: sign([3.1 -2.4 0]) retourne [1 -1 0]
real(<i>var</i>) et imag(<i>var</i>)	Partie réelle, resp. imaginaire, de la <i>var</i> complexe